Transactions & ACID Properties

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. Transactions
2. Transaction Lifecycle
3. Deep Dive into ACID Properties
4. Case Study & Banking
5. Lab Session & Practice

Learning Objectives

By the end of this session, you will be able to:

1. **Master Transaction Control:** Execute multi-step operations using BEGIN, COMMIT, and ROLLBACK in PostgreSQL.
2. **Analyze ACID Properties:** Explain how Atomicity, Consistency, Isolation, and Durability protect data integrity in high-stakes systems (e.g., Banking, AI Agents).
3. **Understand Internal Mechanics:** Describe the role of the **Write-Ahead Log (WAL)** in ensuring Durability and crash recovery.
4. **Handle Complex Logic:** Implement **Savepoints** to manage partial rollbacks in nested business workflows.
5. **Identify Data Anomalies:** Predict what happens when multiple users access the same data simultaneously (a precursor to Concurrency Control).

Transactions

The ”Horror Story”: Life Without Transactions

- **Scenario:** Alice transfers $100 to Bob via a Banking App.
- **Backend Operations:**
  1. UPDATE accounts SET balance = balance - 100 WHERE user = ’Alice’;
  2. – *System Crash / Power Failure / Network Timeout* –
  3. UPDATE accounts SET balance = balance + 100 WHERE user = ’Bob’;
- **The Result:**
  - Alice loses $100.
  - Bob receives nothing.
  - Money vanishes from the system.

Critical Question

How do we guarantee that *either both* steps succeed, or *neither* happens?

What is a Database Transaction?

Definition

A **Transaction** is a logical unit of database processing that includes one or more access operations (INSERT, UPDATE, DELETE).

Key Characteristics:

- **Atomic Unit:** It is treated as a single, indivisible "chunk" of work.
- **Binary Outcome:**
  - **Success:** The transaction is *Committed* (Permanent).
  - **Failure:** The transaction is *Aborted/Rolled back* (No effect).
- **Integrity:** It moves the database from one consistent state to another.

PostgreSQL Motto: *"Either everyone is happy, or nobody moves."*

Transaction Lifecycle

Controlling Transactions in Postgres

In PostgreSQL, we use specific commands to define the boundaries of our work:

- BEGIN: Starts a new transaction block. From this point on, changes are "private" and temporary.
- COMMIT: Makes all changes made since BEGIN permanent and visible to others.
- ROLLBACK: Aborts the transaction and undoes every change made within that block.

```sql
BEGIN;
  UPDATE inventory SET stock = stock - 1 WHERE item_id = 101;
  INSERT INTO orders (item_id, status) VALUES (101, 'PAID');
COMMIT; -- All or nothing happens here
```

Success Pattern

Transaction States: The Journey

A transaction is not just ”open” or ”closed.” It goes through several states:

1. **Active:** The initial state; the transaction is executing SQL statements.
2. **Partially Committed:** The final statement has executed, but the data is not yet safely on disk.
3. **Committed:** Success! The system guarantees the data is permanent.
4. **Failed:** An error occurred (syntax, constraint violation, or crash).
5. **Aborted:** The DB has been restored to its state prior to the transaction.

[State Transition Diagram: Active → Partially Committed → Committed]
[Active → Failed → Aborted]

Implicit vs. Explicit Transactions

Autocommit Mode (Implicit)

By default, PostgreSQL operates in **autocommit** mode. Every single SQL statement is treated as a transaction that commits automatically if it succeeds.

Explicit Transactions

You manually group multiple statements using BEGIN...COMMIT.

Why go Explicit?

- **Business Logic:** Ensuring consistency across multiple tables (e.g., Order + Payment).
- **Performance:** One large COMMIT is significantly faster than 1,000 small ones (less disk I/O overhead).

Postgres-Specific: The "In Failed Transaction" State

In Postgres, if one command fails, the transaction is "poisoned." No further commands are accepted until you ROLLBACK.

```sql
BEGIN;
  -- This valid update will be LOST
  UPDATE film SET rental_rate = 0.99 WHERE rating = 'G';
  -- ERROR: Language ID 999 does not exist (FK violation)
  UPDATE film SET language_id = 999 WHERE title = 'ACADEMY DINOSAUR';
  -- This valid update will be IGNORED
  UPDATE film SET rental_rate = 4.99 WHERE rating = 'R';
COMMIT;
```

Result:

- Because step 2 failed, the entire block is aborted.
- **Verification:** Run SELECT rental_rate FROM film WHERE rating = 'G';
- You will see the old prices. Postgres prioritizes **Atomicity** over partial success.

Live Lab Prep: Observing Isolation

Open two separate Query Tool windows (Session A and Session B) in pgAdmin or psql:

Session A (The Manager):

```sql
BEGIN;
-- Increasing prices for all Action movies
UPDATE film SET rental_rate = 9.99
WHERE film_id IN (SELECT film_id FROM film_category WHERE category_id = 1);
-- DO NOT COMMIT YET!
```

Session B (The Customer):

```sql
-- Checking prices for Action movies
SELECT title, rental_rate FROM film
WHERE film_id IN (SELECT film_id FROM film_category WHERE category_id = 1);
-- Question: Do you see 9.99 or the old price (e.g., 0.99/2.99)?
```

Conclusion: Session B sees the *old* data because A's changes are **Isolated**. This prevents the customer from seeing "temporary" price hikes that might be rolled back.

Deep Dive into ACID Properties

The ACID Properties

To guarantee reliability, every DBMS transaction must adhere to four properties:

- **A - Atomicity:** "All or nothing."
- **C - Consistency:** "From one valid state to another."
- **I - Isolation:** "Concurrent transactions don't interfere."
- **D - Durability:** "Once committed, it stays committed."

Why ACID matters for AI?

If your AI training pipeline or agent logs results to a database that isn't ACID-compliant, a system crash could corrupt your model's history or metadata.

Atomicity: The ”All or Nothing” Rule

- A transaction is an **indivisible** unit of work.
- Even if a transaction has 100 UPDATE statements and the 100th one fails, the previous 99 must be undone.

The Mechanism: PostgreSQL uses **Undo logs** or **MVCC (Multi-Version Concurrency Control)** to track changes. If a rollback occurs, Postgres simply ignores the ”new versions” created during the transaction.

Atomicity in Crashes

What if the server loses power *during* a transaction?

- Upon restart, Postgres performs **Crash Recovery**.
- It looks at the logs: Any transaction that didn't reach the COMMIT state is automatically rolled back.
- **Result:** No "half-finished" data ever exists in your tables.

Consistency: Maintaining the Rules

Consistency ensures that a transaction takes the database from one **valid state** to another, according to all defined rules.

Database Rules include:

- **Constraints:** Primary Keys, Unique, Foreign Keys.
- **Check Constraints:** e.g., $\text{balance} >= 0$.
- **Triggers:** Custom business logic code.

If a transaction violates any of these, the DBMS will reject the change and maintain consistency.

Consistency: Enforcing Business Rules

In dvdrental, we must ensure that data remains logical. For example, a movie cannot have a rental duration of zero or a negative replacement cost.

```sql
ALTER TABLE film
ADD CONSTRAINT min_replacement_cost
CHECK (replacement_cost >= 1.00);

BEGIN;
  UPDATE film
  SET replacement_cost = -5.00
  WHERE title = 'ACADEMY DINOSAUR';
COMMIT;
```

Adding a Business Rule

Outcome: The DB rejects the update. The database stays in a **consistent state** where all films have a positive replacement value.

Isolation: Working in Private

In a real-world system, thousands of transactions run at the same time. **Definition:**

Isolation ensures that the execution of one transaction is not visible to other concurrent transactions until it is committed.

- Without Isolation, we get **"Dirty Reads"** (reading uncommitted, "fake" data).
- **AI Impact:** Parallel data processing scripts might read inconsistent data, leading to incorrect feature engineering.

The 4 Standard Isolation Levels

According to the SQL Standard, there are levels of strictness:

1. **Read Uncommitted:** (Not supported by Postgres - it defaults to Read Committed).
2. **Read Committed:** You only see data that was committed before your query started.
3. **Repeatable Read:** You see a "snapshot" of the data from when your transaction started.
4. **Serializable:** The strictest. Transactions behave as if they ran one after another.

Durability: The ”No-Eraser” Rule

Once a transaction is committed, it remains committed–even in the event of a total system failure (power outage, OS crash).

How is this possible? The DBMS doesn’t wait to write the data to the main database file (which is slow). Instead, it writes to a **Transaction Log** first.

Critical Thinking

Question: Why do you think writing to a log is faster than writing directly to the main database files?

Write-Ahead Logging (WAL)

Postgres uses a technique called **WAL** to ensure durability.

1. Every change is first written to a log file (pg_wal) in a **sequential**, fast way.
2. The log is "flushed" to the disk.
3. Only **after** the log is safe on disk does the COMMIT finish.

The Guarantee

Even if the database files are corrupted, Postgres can "replay" the WAL logs to reconstruct the data.

- **Why must WAL be written sequentially instead of randomly?**
- **In your opinion, what is the "trade-off" of Durability?**

ACID Summary Table

Property	Key Word	Failure if missing...
Atomicity	All or Nothing	Partial data updates
Consistency	Rules/Integrity	Corrupted logic/constraints
Isolation	Privacy/Concurrency	Race conditions
Durability	Persistence	Data loss after crash

Why should AI Engineers care about ACID?

- **Reproducibility:** If your data changes mid-training due to poor isolation, your results aren't reproducible.
- **Data Pipelines:** Moving Petabytes of data requires atomic batches. If a 1TB transfer fails at 99
- **Model Serving:** When updating model weights in a DB, you need Durability to ensure the "winning" model isn't lost on restart.

Case Study & Banking

Case Study: The Banking Standard

In Fintech, "close enough" is not an option. Consider the **Double Spending** problem:

- **Scenario:** A user has $100 and tries to send $100 to two different people at the exact same millisecond.
- **The Atomic Solution:**
  1. Transaction A starts to process the first transfer.
  2. Transaction B starts.
  3. Because of **Atomicity** and **Isolation**, the DB locks Alice's row.
  4. Transaction A completes (Balance: $0).
  5. Transaction B wakes up, sees $0, and is **REJECTED** by a Consistency rule ($\text{balance} >= 0$).

Result

The bank's total assets remain balanced. Without ACID, the bank would literally lose money to "phantom" transactions.

Interactive: How ACID Saves the Bank

Let's analyze our **Double Spending** example. Which ACID property is responsible for which safety measure?

The Banking Scenario	The ACID Hero
The transfer must either complete both legs (Subtract & Add) or none.	*? (Guess the letter)*
The account must never go below $0 (Constraint: $\text{balance} >= 0$).	*? (Guess the letter)*
Transaction B must wait while A is updating the same account.	*? (Guess the letter)*
Even if the city loses power right after "Success", the money is there.	*? (Guess the letter)*

Discussion Guide for AI Students

- **Hint:** If we remove **I (Isolation)**, could two people spend the same $100?
- **Hint:** If we remove **A (Atomicity)**, where does the money go during a crash?

The Trade-off: Throughput vs. ACID

Why don't we just make everything "Serializable" (the strictest level)?

- **Locking Overhead:** Ensuring Atomicity and Isolation requires "Locks."
- **Latency:** Waiting for the WAL log to write to disk (Durability) takes time (milliseconds).
- **AI Scale Problem:** When training on billions of rows, strict ACID on every row update would make training take years.

The Strategy

Use **Batch Transactions**. Instead of 1 million individual transactions, group them into batches of 10,000. This balances safety with high-speed performance.

Lab Session & Practice

Lab 01: The Rental Refund Crisis

Scenario: A customer returns a damaged DVD. We need to:

1. Update the rental status to 'returned'.
2. Charge a damage fee to the payment table.
3. If any step fails, no data should change.

```sql
-- Find a rental to test with
SELECT rental_id, customer_id FROM rental LIMIT 1;
```

Initial Check

Task

Write a transaction that updates the rental and inserts a payment. We will intentionally "break" the second part to test Atomicity.

Exercise 1: Testing the "All or Nothing"

Run this script and observe what happens to the rental record:

```sql
BEGIN;
  -- Step 1: Mark as returned
  UPDATE rental SET return_date = NOW() WHERE rental_id = 2;

  -- Step 2: Intentional Error (Invalid payment_id or Column)
  INSERT INTO payment (customer_id, staff_id, rental_id, amount,
    payment_date)
  VALUES (459, 1, 2, 99.99, 'WRONG_DATE_FORMAT');

COMMIT;
```

Check the result:

```sql
SELECT return_date FROM rental WHERE rental_id = 2;
```

Is the return_date updated? Why not?

Savepoints: Partial Rollbacks

Sometimes, in a long transaction, you don't want to lose *everything* if a small, non-critical part fails.

- SAVEPOINT name: Creates a marker.
- ROLLBACK TO name: Undoes work only back to that marker.

```sql
BEGIN;
  INSERT INTO customer (first_name, last_name, email, address_id, store_id)
  VALUES ('AI', 'Student', 'ai@edu.vn', 1, 1);

  SAVEPOINT user_created;

  -- Try an experimental feature (may fail)
  INSERT INTO special_offers (customer_id, code) VALUES (9999, 'FREE');

  -- If it fails:
  ROLLBACK TO user_created;

  -- The user is still in the "sandbox", ready to be committed!
COMMIT;
```

Exercise 2: The ”Flexible” Staff Task

Create a transaction that:

1. Registers a new staff member.
2. Sets a SAVEPOINT.
3. Tries to assign them to a non-existent store_id.
4. When it fails, **Rollback to Savepoint** and assign them to Store 1 instead.
5. COMMIT everything.

Goal: Ensure the staff member is created even if the initial store assignment was wrong.

Durability: Where is the Log?

Postgres ensures your transaction is "on disk" via the **Write-Ahead Log (WAL)**.

- **On Windows (Default Installation):**

```
# Typically located in:
C:\Program Files\PostgreSQL\[VERSION]\data\pg_wal

```

- **On Linux / Docker:**

```
/var/lib/postgresql/data/pg_wal

```

- These 16MB binary files store every change before it is applied to the main data files.

The "Crash" Test

1. Start a transaction and COMMIT a change. 2. Open Task Manager and **End Task** for all 'postgres.exe' processes (simulating a crash). 3. Restart the service. Your

Session Summary: The ACID Shield

- **Transaction:** A logical container for SQL commands. It's the "Undo" button for your database.
- **Atomicity:** No partial updates. Ever.
- **Consistency:** The database is a set of rules; transactions must obey them.
- **Isolation:** What happens in a transaction, stays in a transaction (until Commit).
- **Durability:** The WAL is your insurance policy against power failure.

Next Session: *Concurrency Control - How to handle 10,000 people buying 1 ticket at the same time.*

Q&A and Final Challenge

- **Q&A:** Any questions on ACID or $\mathrm{dvdrental}$ lab?
- **Challenge:**
  - Research the $\mathrm{ANALYZE}$ command vs. VACUUM in Postgres.
  - How do they relate to transaction history?